

BreachMark research playbook

How to turn BreachMark runs into a study that holds up: from the question, through the experiment, to a paper. Written for a first research project; every step has the exact command or click.

1. What you can and cannot claim

BreachMark answers one family of questions: *given a piece of real code, how well does model M, prompted with strategy S, judge whether it contains a vulnerability, and what changes that?*

You can claim things about **detection accuracy under controlled conditions**: which model, which prompt, with or without the CWE hint, on which dataset, at what cost.

You cannot claim that a model is a good or bad *security tool*. The benchmark asks about one function or file at a time with a known answer. Real review involves whole repositories, no ground truth and a developer in the loop. Say this in your limitations section and reviewers will respect it.

2. Research questions you can answer with this tool

Pick one primary question and at most two secondary ones. A focused paper beats a sprawling one.

ID	Question	Why it is new
RQ1	How much does telling the model the CWE improve detection? (hinted vs blind)	The VulnSage paper and most others always give the hint. Nobody has isolated its effect.
RQ2	Does structured reasoning (CoT, Think, Think & Verify) help more when the hint is removed?	Interaction between prompt and hint has not been studied.
RQ3	Do results transfer across datasets and languages? (VulnSage C/C++ vs SVEN C/C++ vs SVEN Python)	Most papers use one dataset. Cross-dataset agreement is evidence a finding is real.
RQ4	Have 2026 models (commercial and open) improved over the 2024 models in the paper?	The paper tested ten local models up to 70B; no GPT, Claude or Gemini.
RQ5	Does a majority vote of cheap models beat the best expensive one?	BreachMark computes this; the paper did not.
RQ6	What is the cost per correct detection?	Rarely reported; practitioners care.

RQ1 is the recommended primary question: cheapest to run, clearest to explain, and the tool was built for it.

3. Experimental design

Factors you vary

- Model (5 to 10 of them; see section 5)
- Prompt strategy (baseline, cot, think, think_verify)

- Hint (hinted, blind)
- Dataset / language (vulnsage C/C++, sven C/C++, sven Python)

Things you keep fixed, and report

Setting	Value	Where
Temperature	0	run form, Advanced
Max output tokens	4096	run form, Advanced
Input limit	40,000 characters, policy "truncate" (or 0 for models with a large context)	run form, Options
Ollama context window	16,384 or higher, same for all local models	run form, Advanced
Sample selection	same filters, same shuffle seed, same limit for every run	run form, Dataset slice
Judge	off (so parsing is identical across models)	run form, Advanced

The design is **paired**: every model answers the same questions. That is what makes the statistics in section 7 valid and the comparisons fair.

Sample size. The full bundled data is 1,396 samples; both variants and four strategies means 11,168 calls per model per hint condition. For a paper, run the full set for your primary question. For secondary questions, a fixed random subset of 300 samples (seed 1) per model is acceptable if you report the confidence intervals. Never compare a 40-sample run with a full run.

Repeats. Temperature 0 is not perfectly deterministic on most APIs. Repeat one full configuration three times for one model and report the spread. If the spread is small (under 1 point), say so and run the rest once; if not, run everything twice and average.

4. Protocol, step by step

Step 0. Set up a lab notebook. A plain text file `notebook.md` in the project folder. Every run gets a line: date, run id, model string, provider, hint, filters, seed, settings, anything unusual. Appendix A has the template. Reviewers and your future self will thank you.

Step 1. Pilot (one evening). For each candidate model, a 40-sample hinted run with baseline and `think_verify`. Purpose: confirm the connection works, the model follows the VERDICT format (ambiguous rate below 10%), latency is bearable, and cost matches the estimate. Models that fail the format even with `think_verify` get the judge option turned on, which you then record.

Step 2. Primary runs. For each model, two runs with identical settings except the Blind checkbox:

```
python -m breachmark run --provider openai --model qwen/qwen3-32b \
  --strategies baseline cot think think_verify --seed 1 --concurrency 1 \
  --max-input-chars 40000 --name "qwen3-32b hinted"

python -m breachmark run --provider openai --model qwen/qwen3-32b \
  --strategies baseline cot think think_verify --seed 1 --concurrency 1 \
  --max-input-chars 40000 --name "qwen3-32b blind" --blind
```

Name runs `<model> <hinted|blind>` consistently. The Compare page and the standings then read naturally.

Step 3. Export immediately. When a run completes: run page → Export CSV, and save it as `exports/<run-name>.csv`. Also copy `breachmark.db` to a backup folder weekly. CSV exports are what you publish with the paper.

Step 4. Secondary runs. The dataset/language question needs no extra runs if Step 2 used all data: the run page breaks results down by dataset and language. Ensembles need no extra runs either: the Compare page computes the majority vote across the runs you tick. The variance check (section 3) is three extra runs of one configuration.

Step 5. Analysis. Section 7. Do it in a notebook or script that reads the CSV exports, so every table in the paper can be regenerated with one command.

Step 6. Write. Section 9.

5. Choosing models

Aim for coverage, not quantity. A good set of six:

Role	Example	Why
Code-specialised open	qwen2.5-coder:7b or qwen3-coder	The paper's best family
General open, large	llama-3.3-70b or llama-4-scout	General model, strong baseline
Reasoning model	deepseek-r1 or gpt-oss-120b	Tests whether built-in reasoning replaces prompting
Commercial, cheap	gpt-4o-mini or gemini-2.5-flash	What most people would actually use
Commercial, frontier	claude-sonnet or gpt-5 class	Does money buy accuracy?
Small local	a 7B or smaller model via Ollama	What runs on a laptop

Free tiers (Groq, Gemini) cover the first four; the frontier model costs real money, so run it last and only if the budget allows. Record the exact model string and the date; cloud models change behind the same name.

6. Data management

- One folder per study: `breachmark.db`, `notebook.md`, `exports/`, `analysis/`.
- Never delete or rerun a run to "clean it up". Create a new run and note why in the notebook.

- Keep failed and ambiguous answers in the data; report their rates. They are results too.
- When you publish, release the CSV exports, the notebook, the analysis script and the BreachMark commit hash you used (`git rev-parse HEAD`). That is full reproducibility with no extra work.

7. Analysis

BreachMark reports per run and strategy: accuracy with a 95% Wilson interval, recall on vulnerable code, specificity on patched code, precision, F1, pair-correct rate and ambiguous rate. Use them as follows.

Headline metric: pair-correct rate. A sample counts only if both the vulnerable and the patched version were judged correctly. It is the strictest, hardest-to-game number and the one PrimeVul recommends. Report accuracy alongside it for comparison with the VulnSage paper.

The hint gap. For each model and strategy, $\text{gap} = \text{hinted accuracy} - \text{blind accuracy}$. Then look at *where* the points went: did blind recall fall (the model stopped seeing bugs) or did blind specificity fall (it started flagging everything)? The confusion matrices answer that, and the two stories mean different things.

Is a difference real? Use McNemar's test. Both conditions answered the same questions, so compare them pairwise: count questions right under A but wrong under B, and vice versa. Appendix B has a script that reads two export CSVs and prints the test. Report the p-value and the number of discordant pairs. With 2,000+ answers per condition, differences of 2 to 3 points are usually significant; with 80 answers, 10 points may not be.

Per-CWE results. Only interpret cells with at least 20 answers. Report the heatmap for the full runs and say which cells you excluded.

Ambiguous answers. Count them as wrong in the main tables (BreachMark does). Also report the ambiguous rate separately; a model that refuses to commit is behaving differently from one that is wrong.

Cost. Enter the provider's prices in the run form and BreachMark computes cost per run. Divide by the number of correct pair verdicts to get cost per correct detection.

8. Threats to validity (write these down before you start)

- **Training-data contamination.** All bundled vulnerabilities are public and may be in a model's training data. The VulnSage paper found reasoning rather than recall drove results, but you cannot prove it for your models. Mitigate: report results by year; note that blind mode removes the easiest memorisation cue.
- **Truncation.** Long samples are shortened. Report the share of truncated inputs per run and, if large, rerun with a bigger limit or exclude them.
- **Prompt sensitivity.** Wording changes results. Use the built-in prompts unchanged, or publish yours.
- **Non-determinism.** See the repeat protocol in section 3.
- **Label noise.** VulnSage ships a noise estimate per sample; SVEN does not. Report results for low-noise samples (noise 20% or less) as a robustness check.
- **Verdict parsing.** A model may reason correctly and still fail to emit the VERDICT line. Report the ambiguous rate and, if you use the judge, say so and report how often it was invoked.

- **Dataset construction.** You did not build the datasets; their authors' choices (which CVEs, which functions) shape your results. Cite them and describe their construction briefly.

9. Writing it up

A workshop paper is 4 to 6 pages. Suggested structure:

1. **Introduction:** LLMs are being used for security review; benchmarks usually tell the model what to look for; you measure what that hint is worth (one paragraph each). End with contributions as bullets.
2. **Background and related work:** VulnSage (dataset and prompts), SVEN (dataset), PrimeVul (paired evaluation, the contamination problem), VulDetectBench and SecLLMHolmes (LLM limits), CyberSecEval (model safety, different goal). Two paragraphs.
3. **Method:** datasets, the paired vulnerable/patched design, prompts (put the full text in an appendix), hinted vs blind, models and settings (the table from section 3), metrics, statistics.
4. **Results:** one table per research question, one figure (the CWE heatmap or a hinted-vs-blind bar chart), two or three sentences of interpretation per table. Report confidence intervals and p-values.
5. **Discussion:** what the gap means for people using LLMs in review; which failure mode dominates; whether reasoning prompts compensate.
6. **Threats to validity:** section 8, condensed.
7. **Conclusion and artifact:** link to the repository, the exports and the commit hash.

Venues that fit student work: LLM4Code (workshop at ICSE), the MSR data and tool showcase, IEEE SecDev, AISec (workshop at CCS), national conferences. Post to arXiv first; a faculty member must endorse a first-time submitter in cs.CR or cs.SE, and you want a faculty co-author anyway.

Timeline for one person, evenings and weekends: pilots 1 week; primary runs 2 to 3 weeks (they run in the background); analysis 1 week; writing 2 weeks; one week of slack. About two months.

10. Attribution and ethics

Cite VulnSage (Zibaeirad & Vieira, 2025) for the dataset and prompting strategies, SVEN (He & Vechev, 2023) for the second dataset, PrimeVul (Ding et al., 2024) if you import it, and BreachMark as the harness. Be precise about what you built (the harness, blind mode, the experiment) and what you reused. There is no human-subjects component; no ethics approval is needed. Do not publish API keys in exports or notebooks.

Appendix A: lab notebook template

```
## 2026-10-15 run 7 qwen/qwen3-32b (Groq) hinted
filters: dataset=all, seed=1, limit=None strategies: all four
settings: temp 0, max_tokens 4096, input 40000 truncate, concurrency 1, judge off
notes: 3 rate-limit retries around 21:40, no failures. 11,168 calls, 7h12m.
export: exports/qwen3-32b-hinted.csv
```

Appendix B: McNemar's test on two exports

Reads two BreachMark CSV exports of runs that answered the same samples (same filters and seed), compares them strategy by strategy, and prints the discordant counts and p-value. Needs `pip install scipy`.

```
import csv, sys
from scipy.stats import binomtest

def load(path):
    out = {}
    with open(path, newline="", encoding="utf-8") as f:
        for r in csv.DictReader(f):
            if r["status"] == "done":
                out[(r["commit_hash"], r["variant"], r["strategy"].split("@")
[0])] = int(r["correct"])
    return out

a, b = load(sys.argv[1]), load(sys.argv[2])
for strategy in sorted({k[2] for k in a}):
    keys = [k for k in a if k[2] == strategy and k in b]
    a_only = sum(1 for k in keys if a[k] == 1 and b[k] == 0)
    b_only = sum(1 for k in keys if a[k] == 0 and b[k] == 1)
    n = a_only + b_only
    p = binomtest(a_only, n, 0.5).pvalue if n else 1.0
    acc_a = sum(a[k] for k in keys) / len(keys) if keys else 0
    acc_b = sum(b[k] for k in keys) / len(keys) if keys else 0
    print(f"{strategy:13s} n={len(keys):5d} A={acc_a:.3f} B={acc_b:.3f} "
          f"A-only-right={a_only:4d} B-only-right={b_only:4d} p={p:.4f}")
```

Usage: `python mcnemar.py exports/qwen3-32b-hinted.csv exports/qwen3-32b-blind.csv`

Appendix C: results table template

Model	Strategy	n	Hinted acc. [95% CI]	Blind acc. [95% CI]	Gap	p	Hinted recall / spec.	Blind recall / spec.	Ambiguous (h/b)
-------	----------	---	-------------------------	------------------------	-----	---	--------------------------	-------------------------	--------------------

One row per model and strategy. Bold the largest gap per model. Add a "pair-correct" version of the same table in the appendix.